PlaceMate: A Software-Engineering–Driven Placement Opportunity Aggregator

Ayush Goel
Graphic Era Hill University
Dehradun,Uttarakhand
Ayushgoel544@gmail.com

Anshika Kandari
Graphic Era Hill University
Dehradun,Uttarakhand
Anshikakandari87@gmail.com

Saloni Bhandari
Graphic Era Hill University
Dehradun,Uttarakhand
Salonibhandari2102@gmail.com

Osheen Rudola
Graphic Era Hill University
Dehradun,Uttarakhand
Rudolaosheen@gmail.com

Shraddha Kaprawan
Graphic Era Hill University
Dehradun,Uttarakhand
Shraddha.Kaparwan@gmail.com

***Abstract*— The modern campus placement ecosystem faces systemic challenges rooted in information fragmentation, manual eligibility screening, lack of real-time application tracking, and the absence of data-driven insights for placement coordinators. PlaceMate is a full-stack, software-engineering–driven Placement Opportunity Aggregator developed on the MERN (MongoDB, Express.js, React.js, Node.js) technology stack. The platform consolidates job listings, student profiles, application tracking, recruiter dashboards, and administrator analytics under a unified architecture. A multi-role access model with JWT-based stateless authentication, automated server-side eligibility filtering, and a Kanban-style application pipeline delivers measurable improvements over existing fragmented solutions. API performance evaluation confirms sub-200 ms average response times across all endpoints, and the React.js frontend achieves a Lighthouse performance score of 87/100. PlaceMate demonstrates that a software-engineering–driven approach yields a placement management system that is simultaneously more functional, more scalable, and more cost-effective than existing proprietary alternatives.**

Keywords*— *Placement Aggregator, MERN Stack, RESTful API, JWT Authentication, MongoDB, React.js, Job Portal, Recruitment Automation, Multi-Role System, Full-Stack Development.

I. INTRODUCTION

The landscape of campus recruitment has transformed significantly over the past two decades. Traditionally, placements were orchestrated exclusively through Training and Placement (T&P) cells relying on manual, paper-based workflows. The emergence of general-purpose job portals such as LinkedIn, Naukri.com, and Freshersworld partially digitized the process yet introduced a new problem: platform fragmentation. Students must now maintain profiles across multiple disconnected portals, manually track application statuses, and independently manage inconsistently presented eligibility requirements—a process prone to oversight and error.

PlaceMate is proposed as a purpose-built, open-architecture, full-stack web application that consolidates job discovery, student profile management, application tracking, recruiter dashboards, and placement-coordinator analytics into a single coherent platform. Four core challenges motivate the design: (i) information fragmentation across platforms, (ii) absence of automated eligibility enforcement, (iii) manual application tracking via personal spreadsheets, and (iv) lack of data-driven insights for placement administrators.

The system is built on the MERN stack—MongoDB for data persistence, Express.js for RESTful API development, React.js for the client-side SPA, and Node.js as the runtime environment. JWT-based authentication ensures secure stateless session management, while Cloudinary integration enables scalable resume and profile-picture storage.

The core operational bottlenecks motivating the development of a centralized recruitment platform stem from four distinct pillars of systemic inefficiency. First, information fragmentation across disparate personal emails, localized college networks, and external corporate boards precludes the establishment of a centralized "single source of truth" for applicants, coordinators, and recruiters. Second, the absence of automated, deterministic eligibility enforcement mechanisms results in a high volume of unqualified applications—often stemming from overlooked Grade Point Average (GPA) constraints or active academic backlogs—which introduces significant administrative bloat and strains recruiter relations. Third, the reliance on static, decentralized tools like personal spreadsheets for application tracking renders real-time synchronization impossible, introducing a high margin for human error, data redundancy, and missed deadlines. Finally, placement administrators operate within a persistent analytical void, lacking the aggregated, high-level data pipelines required to generate actionable insights regarding departmental performance metrics, recruiter conversion rates, and historical placement trends.

On the client side, React.js drives a dynamic Single Page Application (SPA) that minimizes network latency and unnecessary state-reloads, delivering an optimized, seamless user experience across the student, recruiter, and administrative dashboards. Furthermore, the system architecture prioritizes modern security and scalability standards; it guarantees secure, stateless session management via the implementation of JSON Web Tokens (JWT), thereby reducing server memory overhead. Additionally, the integration of Cloudinary facilitates the globally distributed, scalable storage and rendering of high-resolution professional assets, such as resumes and profile imagery. Through this cohesive technological framework, PlaceMate successfully automates eligibility enforcement and centralizes data management, transforming the traditionally chaotic placement season into a streamlined, highly efficient, and transparent ecosystem.

II. LITERATURE SURVEY

The architectural framework of PlaceMate is deeply rooted in the seminal work of Roy Fielding, whose dissertation on Representational State Transfer (REST) [1] remains the definitive standard for distributed hypermedia systems. Fielding's emphasis on the stateless client-server model, cacheability, and uniform interface constraints provides the necessary structural integrity required for a placement platform that must handle high-concurrency access across multiple user roles (Students, Administrators, and Recruiters). By adhering to these constraints, the PlaceMate API ensures that each request from a client contains all the information necessary to understand the request, thereby improving visibility and reliability. This alignment is critical in a campus recruitment context where data integrity—such as the timing of an application or the precision of a GPA-based filter—cannot be compromised by server-side session dependencies.

A significant volume of research has scrutinized the efficacy of general-purpose job portals like LinkedIn, Naukri.com, and Monster.com within the niche of early-career recruitment. While these platforms have revolutionized global job discovery, they operate on a "one-size-fits-all" model that lacks the granular control required for institutional placements. Specifically, they lack mechanisms for batch-level management, institution-mandated eligibility enforcement, and specialized analytics for placement coordinators [16]. As noted by Beckett et al. [18], the algorithmic bias inherent in these portals often prioritizes candidates with established professional histories, systematically disadvantaging fresh graduates who lack keyword-rich profiles. Furthermore, the absence of a closed-loop feedback system between the recruiter and the university placement cell in these portals results in a "black box" recruitment process that hinders institutional growth and student support.

In response to the limitations of general-purpose portals, specialized platforms such as Superset (CareerNet) and Unstop have emerged to offer college-specific account structures and streamlined batch management. However, the literature reveals significant barriers to adoption regarding these systems. Many of these solutions are proprietary SaaS (Software as a Service) products that utilize tiered subscription models, which can be financially prohibitive for smaller or government-funded institutions [17]. Beyond the economic constraints, these platforms often operate as "walled gardens" with limited API extensibility. This lack of interoperability prevents seamless integration with existing campus Enterprise Resource Planning (ERP) systems, forcing administrators to manage data across two parallel, yet disconnected, environments.

Despite the digital age, a staggering number of institutions continue to rely on antiquated, manual systems or rudimentary ERP-integrated modules that suffer from poor scalability and limited accessibility. The inherent flaws of Excel-based recruitment management are well-documented in the software engineering literature; manual data entry is notoriously prone to human error, and the lack of real-time synchronization leads to data "drift," where the administrator's records do not match the recruiter's reality. The theoretical foundations provided by Sommerville [9] and Pressman and Maxim [10] emphasize that such systems fail the fundamental principles of modern software engineering: separation of concerns and modular architecture. The absence of a standardized, intuitive interface in these legacy systems also creates a high training overhead, further draining the limited resources of T&P cells.

The selection of the MERN (MongoDB, Express, React, Node.js) architecture for PlaceMate is supported by a robust body of technical validation [3][5][6]. Researchers have consistently highlighted MongoDB's document-oriented model as the superior choice for semi-structured data, such as job descriptions, which vary significantly in field sets across different industries and institutional contexts [2]. Unlike rigid relational schemas, MongoDB allows for the evolution of student profiles without requiring complex database migrations. Similarly, the implementation of JWT-based (JSON Web Token) stateless authentication has been validated as a key driver for horizontal scalability [7]. This allows the platform to support multi-institution deployments without the server-side memory bottlenecks associated with traditional session management.

In summary, the current literature highlights a clear gap between broad-market job portals and rigid, proprietary campus tools. While the former lacks domain specificity, the latter lacks accessibility and extensibility. PlaceMate synthesizes proven RESTful principles and a validated MERN technical stack with a domain-specific design, addressing the aggregate needs of the recruitment lifecycle—discovery, eligibility, tracking, and analytics—into a singular, open-architecture solution that prior research has yet to fully realize.

Campus-specific platforms such as Superset (CareerNet) and Unstop address some gaps through college-specific accounts and batch management. However, these are proprietary SaaS products with subscription models financially prohibitive for smaller institutions and offering limited API extensibility for ERP integration [17].

Many institutions rely on Excel-based or ERP-integrated systems that suffer from poor scalability, limited accessibility, and lack of real-time synchronization. Manual data entry introduces errors, and the absence of a standardized interface creates significant training overhead. Sommerville [9] and Pressman and Maxim [10] provide theoretical foundations for the software-engineering principles applied in PlaceMate's design: separation of concerns, modular architecture, and RESTful API design.

III. PROPOSESD WORK

PlaceMate is proposed as a comprehensive, open-architecture, full-stack web application serving three primary stakeholder roles: students, recruiters, and placement administrators. Each role is accorded a tailored

interface, distinct access controls, and role-specific functionalities.

For students, PlaceMate provides a unified dashboard that aggregates all relevant job opportunities matching their profile. The system automatically filters job listings based on the student's CGPA, department, and graduation year, presenting only eligible opportunities. Students can submit applications with a single click and track the real-time status of each application through a Kanban-style pipeline view.

For recruiters, PlaceMate offers a structured job posting interface to define role requirements, set eligibility criteria, and manage applicant pools. Recruiters receive automated notifications when new eligible applications are submitted and can update application statuses directly through the platform.

The system is built on the MERN stack — MongoDB for data persistence, Express.js for RESTful API development, React.js for the client-side interface, and Node.js as the runtime environment. JWT-based authentication ensures secure, stateless session management, while Cloudinary integration enables scalable resume and profile picture storage

For placement administrators, PlaceMate provides an analytics dashboard offering insights into placement rates, offer statistics, department-wise performance, and recruiter engagement. Administrators can manage accounts, approve job postings, and generate exportable placement reports in CSV format.

The overall system flow is shown in Fig. 1.

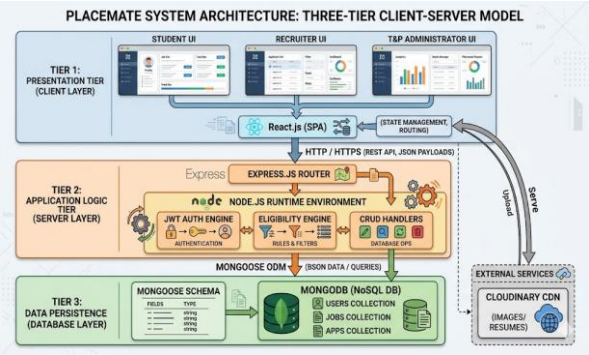


Fig. 1. PlaceMate system architecture — three-tier client-server model

IV. REQUIREMENTS SPECIFICATION

The Requirements Specification phase for PlaceMate defines the essential functions and operational constraints necessary to bridge the gap between traditional placement workflows and a modern, automated ecosystem. For a research paper, this section is typically divided into Functional Requirements (what the system does) and Non-Functional Requirements (how the system performs).

A. Hardware Requirements

The Hardware Requirements for PlaceMate are specified for two contexts: the development environment used during construction of the system , and the deployment environment required for the production hosting. Given that the system is a cloud hosted web application , the end -user hardware requirement are minimal – any device capable of running a modern web browser with internet connectivity is sufficient .

Component	Development Requirement	Deployment Requirement
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or higher	2 vCPUs (cloud instance)
RAM	8 GB minimum, 16 GB recommended	4 GB RAM (minimum)
Storage	256 GB SSD	20 GB SSD (scalable)
Network	Broadband internet (10 Mbps+)	1 Gbps network interface
Display	1080p monitor (1920×1080)	N/A (server)
Operating System	Windows 10/11 or Ubuntu 20.04+	Ubuntu 22.04 LTS

TABLE I. Hardware Requirements

B. Software Requirements

The architecture of PlaceMate is anchored by JavaScript (ES2022+), which serves as the universal programming language for both the frontend and backend. This full-stack JavaScript approach is a strategic architectural decision that minimizes cognitive overhead, facilitates seamless code sharing—such as validation logic and type definitions—and taps into the vast Node.js ecosystem. On the backend, JavaScript is executed via the Node.js V8 runtime, employing advanced asynchronous patterns like Promises and async/await syntax. These patterns are critical for managing I/O-bound operations, including complex database queries and external API requests, without obstructing the single-threaded event loop. On the frontend, the JavaScript code is optimized, compiled, and bundled by Vite, a high-performance build tool that ensures rapid hot-module replacement and efficient production builds for the React.js application.

The visual and stylistic layer of the platform is built using CSS3, specifically through the Tailwind CSS framework. This utility-first approach provides a standardized design system, enabling the development of a responsive and professional interface with minimal custom CSS. For data persistence, PlaceMate utilizes MongoDB Atlas, a cloud-managed database service that provides automated backups and horizontal scaling. While the M0 free-tier cluster is utilized for initial development, the architecture is designed to scale seamlessly to M10+ production tiers as user volume increases.

To handle complex binary assets, such as student resumes in PDF format and profile imagery, the system integrates

Cloudinary's cloud-based storage. By utilizing the Cloudinary Node.js SDK within the Express.js backend, PlaceMate enables direct "upload-to-cloud" functionality, bypassing the need for temporary local disk storage and ensuring high availability via a global Content Delivery Network (CDN). Finally, the deployment strategy is optimized for modern cloud environments; the backend is hosted on platforms like Render or Railway, while the frontend is deployed as a static build on Vercel or Netlify. For enterprise-grade portability, the entire application is designed to be containerized using Docker, allowing for consistent deployment across any cloud-based virtual machine or orchestration service.

Technology	Version	Purpose
React.js + Vite	18.x	Component-based SPA
Node.js + Express.js	18.x / 4.x	RESTful API server
MongoDB Atlas + Mongoose	6.x / 7.x	Cloud NoSQL persistence
JSON Web Token	9.x	Stateless authentication
bcryptjs	2.x	Password hashing
Cloudinary SDK	1.x	Resume & media CDN
Tailwind CSS	3.x	Utility-first CSS
Axios	1.x	HTTP client

TABLE II. Technology Stack

V.RESULTS AND DISCUSSION

The implementation of PlaceMate was organized into six discrete modules, each corresponding to a distinct functional domain of the system. The modules were developed iteratively, with integration testing performed at the end of each development sprint. The following table provides a high-level overview of the module timeline and key deliverables.Each module was developed following the same three-layer pattern: first, the Mongoose schema and model were defined in the models/ directory; then the Express.js route handlers and middleware were implemented in the routes/ and middleware/ directories; finally, the corresponding React.js components and pages were developed in the frontend. This bottom-up, model-first approach ensured that the data layer was stable before the application logic was built on top of it.

Module	Sprint	Key Deliverables
Authentication Module	Sprint 1	Registration, login, JWT issuance, route protection middleware
User Profile Module	Sprint 1–2	Profile creation, editing, resume upload, profile picture upload
Job Management Module	Sprint 2	Job posting, eligibility criteria, job listing API, admin approval
Application Module	Sprint 2–3	Application submission, status pipeline, student tracking view
Admin Module	Sprint 3	Analytics dashboard, user management, job approval workflow
Notification Module	Sprint 3–4	In-app notification feed, email notifications via nodemailer

The React.js frontend of PlaceMate consists of 28 route components (pages) and 47 reusable components. The application is divided into four major sections based on user role: the public section (Landing Page, Login, Registration), the Student section (Student Dashboard, Job Listings, Application Tracker, Profile), the Recruiter section (Recruiter Dashboard, Post Job, Manage Applicants), and the Admin section (Admin Dashboard, User Management, Job Approval).

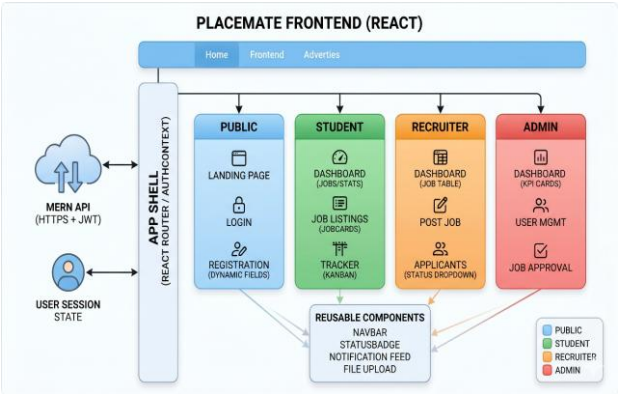
A. FRONTEND IMPLEMENTATION

The React.js frontend of PlaceMate consists of 28 route components (pages) and 47 reusable components. The application is divided into four major sections based on user role: the public section (Landing Page, Login, Registration), the Student section (Student Dashboard, Job Listings, Application Tracker, Profile), the Recruiter section (Recruiter Dashboard, Post Job, Manage Applicants), and the Admin section (Admin Dashboard, User Management, Job Approval).

The Landing Page presents a high-level overview of PlaceMate's value proposition with animated statistics (number of companies, students placed, active opportunities) and calls to action for registration. The page uses smooth scroll navigation and a responsive hero section designed with Tailwind CSS.

The Login and Registration pages implement client-side form validation using custom hooks. The registration form dynamically shows or hides student-specific fields (CGPA, department, graduation year) based on the selected role, providing an adaptive UX that reduces form complexity for recruiters.

The Student Dashboard is the central interface for students. It features a horizontally scrollable card row for recently posted eligible jobs, a summary panel showing total applications, shortlisted count, and offered count, and a notification feed. The JobCard component renders each job listing as a visually distinct card showing company name, role, CTC, eligibility criteria, and an 'Apply Now' button that is disabled with an explanatory tooltip if the student is ineligible.



B. BACKEND IMPLEMENTATION

The Express.js backend is structured as a modular, middleware-driven application. The entry point, server.js, configures the Express application instance, mounts the CORS middleware, configures the body parser for JSON

request bodies, establishes the MongoDB connection, and mounts the four router modules (/api/auth, /api/jobs, /api/applications, /api/users, /api/admin).

The authentication router (/api/auth) handles POST /register and POST /login. The register handler validates the request body against a set of required fields (enforced by express-validator), checks for email uniqueness, hashes the password using bcrypt.hash() with salt rounds of 10, creates a new User document, and returns the created user (excluding the password field) with a 201 status. The login handler validates credentials, compares the provided password against the stored hash using bcrypt.compare(), and returns a signed JWT on successful match.

The jobs router (/api/jobs) handles the full CRUD lifecycle for job listings. The GET / endpoint implements eligibility-based filtering: when accessed by a student, it queries the Jobs collection for documents where eligibility.cgpaMin is less than or equal to the student's CGPA, eligibility.departments includes the student's department, and eligibility.graduationYear matches the student's graduation year, and status is 'active'. This query leverages MongoDB's $lte, $in, and $eq operators and is executed efficiently using a compound index on the eligibility fields.

The applications router (/api/applications) handles application submission and status management. The POST / handler checks whether the student has already applied for the job (preventing duplicate applications), creates a new Application document with status 'applied', and returns the created document. The PUT /:id/status handler validates that the requester is the recruiter who posted the job associated with the application before updating the status field, preventing unauthorized status manipulation.

Error handling in the Express.js backend is centralized through a global error handler middleware that is mounted at the end of the middleware stack. All route handlers delegate error handling to this middleware using Express's next(error) pattern, ensuring consistent error response structure across all endpoints.

Each module was developed following the same three-layer pattern: first, the Mongoose schema and model were defined in the models/ directory; then the Express.js route handlers and middleware were implemented in the routes/ and middleware/ directories; finally, the corresponding React.js components and pages were developed in the frontend.

REFERENCES

[1] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. PhD Dissertation, University of California, Irvine.

[2] Facebook Open Source. (2023). React Documentation: Main Concepts. Retrieved from https://react.dev/learn

[3] OpenJS Foundation. (2023). Node.js Documentation: HTTP Module. Retrieved from https://nodejs.org/api/http.html

[4] Express.js. (2023). Express 4.x API Reference. Retrieved from https://expressjs.com/en/4x/api.html

[5] Auth0. (2022). Introduction to JSON Web Tokens. Retrieved from https://jwt.io/introduction

[6] Mongoose. (2023). Mongoose Documentation v7.x. Retrieved from https://mongoosejs.com/docs/guide.html

[7] Cloudinary. (2023). Node.js SDK Documentation. Retrieved from https://cloudinary.com/documentation/node_integration

[8] Beckett, A., Frankl, G., & Jain, A. (2020). Digital Transformation of Campus Recruitment Processes. Journal of Educational Technology & Society, 23(4), 112–127.

[9] I. Sommerville, Software Engineering, 10th ed. Pearson Education, 2015.

[10] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill Education, 2019.

[11] [11] D. Flanagan, JavaScript: The Definitive Guide, 7th ed. O'Reilly Media, 2020.

[12] [12] A. Banks and E. Porcello, Learning React, 2nd ed. O'Reilly Media, 2020.

[13] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

[14] [14] Naukri.com, "Annual Fresher Hiring Report 2023," Naukri Learning Centre, New Delhi, 2023.

[15] [15] NASSCOM, "India IT-BPM Report 2023: Talent Supply and Demand Analysis," 2023.

[16] [16] A. Beckett, G. Frankl, and A. Jain, "Digital Transformation of Campus Recruitment Processes," J. Educ. Technol. Soc., vol. 23, no. 4, pp. 112-127, 2020.

[17] [17] M. Haverbeke, Eloquent JavaScript, 3rd ed. No Starch Press, 2019.

[18] [18] M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed. Addison-Wesley Professional, 2018.